

Grading and AI-risk review for your UCD Data Analytics group project

Project snapshot

You built a full end-to-end notebook that covers data checks, cleaning, feature work, modelling, and interpretation.

You worked with a training set of **54,000 rows** and a test set of **10,000 rows** (same fields, test includes prices so you can score holdout). After deduping on a full transaction “signature”, you kept **53,992** training rows (you dropped **8** duplicates). You faced very high missingness in **Eircode (~68.7%)** and **Property Size Description (~94.9%)**, so you sensibly treated “Property Size Description” as unusable and focused on location engineering.

You designed a routing-key approach for Eircode. You reduced routing-key missingness from **68.70% to 32.62%** via inference from `Address + County`, and you also flagged invalid routing keys (14 “fake” values) plus ambiguous routing keys (those mapping to multiple counties). That gives you an explainable, local, non-API approach.

You trained a CatBoost regressor on **log(price)** and you ran both a time-based validation split and 5-fold time-series CV. You also ran a proper baseline comparison and a feature ablation study.

Academic grading estimate

You meet Master’s-level expectations on structure, evaluation discipline, and feature reasoning. You also show real “project thinking” (trade-offs, reproducibility, ablation, and error analysis). That pushes you above the typical “good notebook” level.

Estimated mark range: 68–74%

Single best estimate: 72% (around an **A- band**, depending on the module’s grade scale). ¹

That estimate assumes you also submit a solid written report and you package external datasets correctly. If you miss either, the mark can drop fast.

Data cleaning and preprocessing

You made good calls early. You deduped using a full transaction signature, you kept row traceability (`row_id`), and you converted types cleanly (dates, prices, low-cardinality categoricals). You also ran quality summaries that make your later choices defensible.

You handled the two biggest data issues well. You did not try to “pretend” Property Size existed. You engineered location signal from what you had.

Where you can still lose marks: you need to state a clear “cleaning contract” upfront. Example: what you treat as unknown, what you drop, what you impute, and why those choices won’t leak the target.

Feature engineering

You earned marks here. Your routing-key pipeline looks thoughtful and testable. Your `Is_Second_Hand` conversion also reads as a grounded feature with a clear meaning.

You also built two advanced feature blocks: - **Internal market history features** (rolling median log-price by county / routing key, lagged with a shift so the same month does not see itself). - **External macro / density features** (mortgage rates, population, new dwellings, area size).

That breadth shows ambition. It also increases the burden on you: you must prove the external sources, merge logic, and time alignment.

Your own ablation study shows an important (and credit-worthy) finding: **core + external** performed best on the holdout test, while the internal engineered features hurt holdout in your tested setup. That kind of honest result reporting scores well.

Model choice and justification

You picked the right kind of model for your feature mix. CatBoost fits categorical location features well. Log-transforming price also makes sense for a heavy-tailed target.

You still leave marks on the table on tuning. You tuned iterations via early stopping, which helps, but you did not show a proper search over depth, regularisation, or sampling parameters. For a Master's project, graders often expect at least a small, justified tuning pass (even if you keep it light).

Evaluation metrics and validation strategy

You did the most important thing: you treated time properly.

You used: - a **last-12-months validation split**, and - **TimeSeriesSplit (5 folds)** CV.

That aligns with how property markets shift over time, and it avoids optimistic random splits.

You also reported multiple metrics (RMSE, MAE, R^2 , MAPE) in both log-space and euro-space. That helps interpretation.

One risk: your feature ablation used the labelled test set to decide what "works best". If the assignment treats that test set as a final holdout, you should avoid selecting the final model using it. Use CV for selection, then report holdout once at the end.

Code quality and structure

You wrote a clear notebook narrative. You also packaged logic into reusable functions (good sign for engineering discipline). You saved clean exports (`cleaned_full`, `model_ready`) and you kept train/test pipelines aligned.

You still have two "grader pain points": - The notebook does a lot. A short `README` plus a one-command run path would help. - External files matter. If a grader cannot run your notebook from scratch with the submitted zip, they will mark you down hard.

Interpretation of results

You added feature importance, you compared train vs CV vs test, and you listed worst errors. That hits what examiners want: “do you understand what your model learned, and where it fails?”

You can push this further with SHAP (CatBoost supports it), and with segmented error analysis (by county, year, and price band). That tends to score well because it shows insight, not just modelling.

Originality and depth of analysis

Your routing-key inference and your careful time validation move you into “high-quality” territory. Many groups stop at basic cleaning plus a random split. You did more.

Your geospatial area-size idea also signals originality. It needs packaging and justification, but the direction makes sense.

Technical review

You built strong core signals. Your “core cleaned” feature set (county, routing key, not-full-market, second-hand, and time) already predicts well. That matches what we’d expect: location and time drive a lot of price variation.

You then added richer features. That added complexity, and your results show mixed value.

Feature strength

Your strongest features likely come from: - **location categories** (`county_clean`, `area_code_clean`) - **transaction context** (`Not Full Market Price`, `Is_Second_Hand`) - **time** (`sale_year`, `sale_month`)

That matches your feature importance story.

Your weaker features likely come from: - **address keyword flags** (they can help a little, but they stay noisy and brittle) - **some rolling history signals** (useful in principle, but sensitive to missingness and regime shift) - **external merges with missing coverage** (they help only if alignment and coverage work)

Your own ablation suggests a clear action: keep external features, but treat internal engineered features as optional until they prove themselves under CV, not just holdout.

Leakage and correctness

You avoided the most common leak. You did not feed `Price (€)` directly into the feature matrix.

Your rolling median features also look conceptually safe because you lag time (you compute “previous months” medians, not “this month”). That matters.

The bigger “leak-like” risk comes from **model selection on the holdout test**. That does not leak target values into features, but it can still inflate your claimed performance if you tune based on the holdout.

Model appropriateness

CatBoost makes sense. It handles categorical features well and it performs strongly on tabular problems.

Your hyperparameters look reasonable. Your tuning approach (early stopping to pick best iteration ≈ 966) works as a first step.

You can still lift performance and credibility by doing a small, controlled search: - depth: 6 / 8 / 10 - l2_leaf_reg: 3 / 5 / 10 - subsample or bagging_temperature - and maybe loss functions (RMSE vs MAE, depending on what you optimise)

Missing steps that would raise the grade

You already did more than most groups, so the gains now come from tightening and proving.

The fastest “+10–15%” improvements usually come from: 1) **A reproducible run path**. Put every external dataset in the zip, or add a script that downloads and verifies it. Add a one-screen [README](#) with the full run order and expected output files.

2) **CV-based feature selection**. Re-run the ablation using the same TimeSeriesSplit folds. Pick the final feature set using CV mean metrics, then report holdout once.

3) **Better error analysis**. Break errors by year, county, and price percentile. Explain why the worst errors happen (multi-unit sales, development blocks, unusual addresses).

4) **Explain external features properly**. Name each source, state the date coverage, and justify why it should predict prices at transaction time.

5) **One deeper interpretation tool**. Add SHAP summary plus 2–3 local explanations for “typical home”, “expensive outlier”, and “rural property”.

Validation and performance report

You handled evaluation well. You used time-based validation and time-series CV, so your numbers look believable.

What your metrics say

Your summary reports a clean pattern:

- **Train (full fit):** log_RMSE ≈ 0.52
- **CV mean:** log_RMSE ≈ 0.57
- **Holdout test:** log_RMSE ≈ 0.61

That drop signals **moderate overfitting**. It also signals **real learning**, because you beat the baseline clearly.

Your baseline vs model on holdout test shows: - log_RMSE improved from **0.73 $\rightarrow$ 0.61** (a large gain) - euro MAE improved from **$\sim 236k \rightarrow \sim 201k$** (meaningful) - euro R^2 stays low (**~ 0.063**), which fits a dataset with very few property-specific variables (no bedrooms, floor area, etc.)

Extreme prices drive euro-scale errors

Your dataset includes extreme outliers (multi-unit or complex sales). That makes euro RMSE blow up.

You chose the right mitigation by modelling in log space. You can score extra marks by explicitly framing this as: “We optimise log-scale accuracy to handle heavy tails, then we back-transform for interpretation.”

Your ablation result gives you a clear next move

Your holdout ablation shows: - **Core + external features** performs best on holdout log_RMSE (≈ 0.53). - Internal engineered features hurt holdout in your tested setup.

That means you have a straight upgrade path: - pick **core + external** as the final model (unless CV disagrees), and - explain why internal features hurt (missingness, noise, regime change, or redundancy with location categories).

That kind of “we tested it, it didn’t help, so we dropped it” often scores higher than “we added everything”.

AI detection risk and academic integrity

You asked a direct question, so I’ll answer directly.

What policy risk matters most

At University College Dublin ², unauthorised use of generative AI can count as academic misconduct if the module does not explicitly permit it or if you do not acknowledge it. UCD also states module coordinators must record permission for AI use in module descriptors, and unacknowledged AI-generated content can trigger misconduct processes. ³

So the first risk does not come from a detector score. It comes from module rules and disclosure.

How reliable “AI detection scores” tend to be

Most detectors come with strong caveats.

Turnitin ⁴’s own documentation notes false positives can happen, and it avoids surfacing AI scores below a threshold (it has used a 20% cutoff) to reduce false-positive harm. ⁵

OpenAI ⁶ explicitly wrote that “it is impossible to reliably detect all AI-written text”, and it removed its AI classifier due to low accuracy. ⁷

Peer-reviewed surveys also describe the broader technical limits and evaluation pitfalls in AI-text detection, especially under real-world conditions. ⁸

Estimated AI-flag risk for what you uploaded

I can only judge the notebook content you shared (not a separate report).

AI-likelihood score (0–100%): 30%

Why that score stays moderate: - Your notebook mixes polished sections with rough, human “project notes”. That pattern looks like real group work, not fully AI-generated prose. - Your work includes detailed, domain-specific steps (routing-key inference diagnostics, time-series evaluation, ablation). That looks like real modelling work, not generic filler.

What could raise suspicion: - If your final written report reads uniformly “perfect”, generic, and citation-free while the notebook reads more informal, the style mismatch can look like AI assistance. - If you cannot show provenance for external datasets or key modelling choices, reviewers may infer you “assembled” rather than “worked”.

How to reduce AI suspicion without “gaming” anything

You reduce risk by increasing evidence of real work, not by trying to trick detectors.

Do these instead: - Add short “decision notes” beside each major choice (why you dropped size, why you chose log target, why time split). Keep the tone consistent across the whole submission. - Cite every external dataset and explain the join keys and time coverage. - Keep a short appendix of experiments: “we tried X, it failed because Y”. - If the module allows AI help, add a one-paragraph disclosure: what you used it for (e.g., grammar, refactoring ideas) and what you wrote and validated yourselves. That aligns with UCD’s emphasis on acknowledgement when permitted. ⁹

Final verdict

Your project already sits near a First-class threshold for many UCD modules (≥ 70 often maps to an A-band, depending on the scale used). ¹

You can still add 10–15 marks by tightening reproducibility, feature selection discipline, and interpretation depth.

Strengths

- You built an end-to-end pipeline with strong time-aware evaluation and a clear baseline.
- You engineered location signal in an explainable way despite very high Eircode missingness.
- You ran ablation and error analysis, and you reported mixed results rather than hiding them.

Weaknesses

- You rely on external files and geospatial assets. A missing file can break the run and cost marks.
- You did not use CV to choose the final feature set, even though ablation suggests a simpler model works better.
- You stopped tuning early. You tuned iterations, but you did not justify or test key CatBoost regularisation knobs.

Predicted grade and AI risk

Predicted grade: 72% (range **68–74%**, conditional on a good report and a runnable submission zip).

AI-flag risk: ~30% for the notebook content you uploaded.

Fix first before submission

- Make the project fully runnable from a clean machine (zip includes external datasets, or script downloads them, plus a `README` run order).
- Re-run ablation under TimeSeriesSplit CV and choose the final feature set using CV, not the holdout test.
- Add SHAP + segmented error analysis (county/year/price band) and turn that into 1–2 strong report pages.
- Add proper source referencing for every external dataset and a clear statement on AI tool use, aligned with module rules. 9

1 Understanding Grades - UCD Current Students

https://www.ucd.ie/students/exams/gradingandremediation/understandinggrades/?utm_source=chatgpt.com

2 3 4 9 Student Conduct - UCD Student Engagement, Conduct, Complaints and Appeals

https://www.ucd.ie/secca/studentconduct/?utm_source=chatgpt.com

5 6 AI writing detection model – Turnitin Guides

https://guides.turnitin.com/hc/en-us/articles/28294949544717-AI-writing-detection-model?utm_source=chatgpt.com

7 New AI classifier for indicating AI-written text | OpenAI

https://openai.com/blog/new-ai-classifier-for-indicating-ai-written-text?o=8874%2Fembed&utm_source=chatgpt.com

8 AI-generated text detection: A comprehensive review of methods, datasets, and applications - ScienceDirect

https://www.sciencedirect.com/science/article/pii/S1574013725000693?utm_source=chatgpt.com